ClaimSage — Architecture & Learning Guide

This document explains **how ClaimSage works and why**, aimed at someone learning RAG systems. Read it top-to-bottom; each section builds on the last.

---

1. What problem does this solve?

You upload large insurance documents (a 100-page policy PDF, an Excel schedule of benefits, claim emails) and ask plain-English questions like *"Is knee surgery covered and what's the waiting period?"* ClaimSage answers **using only your documents**, and shows **which page it got the answer from**.

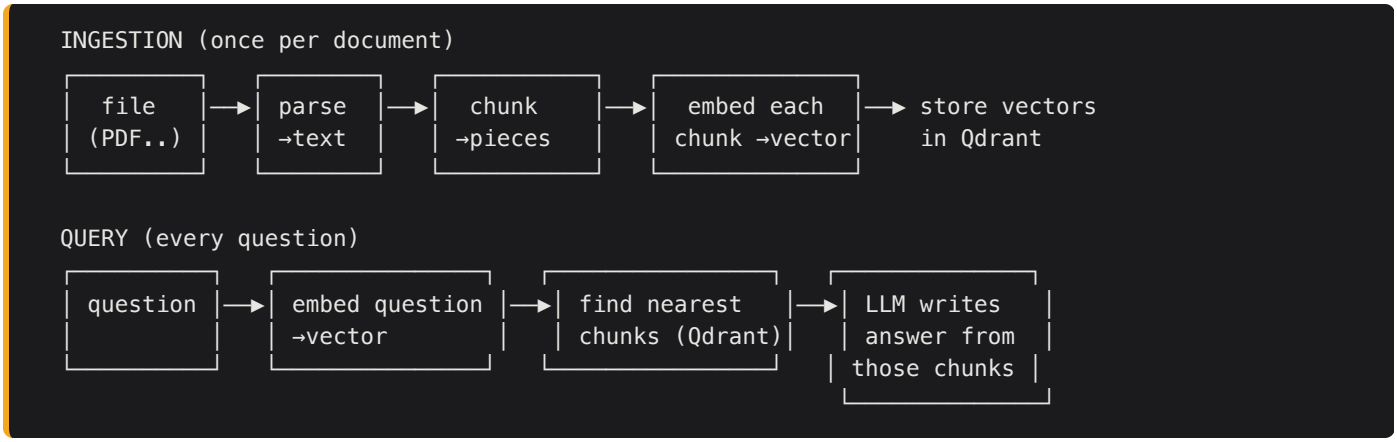
The naive approach — "paste the whole 100-page policy into an LLM and ask" — fails because:

- documents are too big to fit in the model's context window, and
- it's slow and expensive to send everything on every question.

The solution is **RAG (Retrieval-Augmented Generation)**.

---

2. RAG in one picture



Key insight: the LLM never reads the whole document. Retrieval finds the handful of relevant chunks first, and the LLM only summarizes *those*. That's why a small/free model is "enough" — it's doing the easy part.

Two model types are involved (they are different things):

- an **embedding model** turns text into a vector (a list of numbers) so we can measure "similarity" — used to *find* relevant chunks.
- a **chat/LLM model** writes the natural-language answer — used to *compose* it.

---

3. The moving parts

Component	Tech	Role
API server	FastAPI (Python)	HTTP endpoints the frontend calls
Ingestion	LlamaIndex	parse → chunk → embed → store
Vector store	**Qdrant** (Docker)	stores chunk vectors; does similarity search
Registry	**SQLite**	tracks sessions + document ingestion status
Chat model	`google/gemma-4-26b-a4b-it:free` (OpenRouter)	writes answers
Embedding model	`nvidia/llama-nemotron-embed-vl-1b-v2:free` (OpenRouter)	text → 2048-dim vector

Everything runs free: OpenRouter's free models + self-hosted Qdrant + local SQLite.

Why two databases? Qdrant is great at "find similar vectors" but shouldn't track bookkeeping like "is this upload done processing?". SQLite handles that relational bookkeeping; Qdrant handles vector search. Different jobs.

---

4. Ingestion pipeline (walk-through)

Code path: `routes_upload.py` → `ingestion.py` → `parsers.py` + `indexing.py`

1. **Upload** (`POST /api/documents`). The file is saved to disk and a `Document` row is created in SQLite with status `processing` . The response returns *immediately* — we don't make you wait for embedding.
2. **Background task** (`ingest_document`) does the slow work off the request:
    - **Parse** (`parsers.py`): each format becomes LlamaIndex `Document` objects carrying **metadata** — `source_filename` , and a location key (`page_label` for PDF, `sheet_name` + `row_number` for Excel, `email_field` for email). This metadata is what makes citations possible later.
        - PDF → one Document per page (via PyMuPDF) so we know the page number.
        - Excel → one Document per row (columns flattened to text) so a policy schedule row is never split in half.
        - Email → headers + body, and attachments are recursively parsed.
    - **Chunk** (`indexing.build_node_parser`): long text is split into ~512-token `Node` s with 50-token overlap (overlap avoids cutting a sentence's meaning across a boundary). Short Excel rows stay whole.
    - **Embed + store** (`index.insert_nodes`): each node is embedded (OpenRouter) and upserted into the Qdrant collection, tagged with `claim_id` (= the session id).

3. **Status update**: the SQLite row flips to `ready` (with `chunks_indexed`) or `failed` (with an error message). The frontend polls `GET /api/documents/{id}/status` to know when it can start asking questions.

> **Why background processing?** Embedding a big PDF takes seconds to minutes. Doing it inside the HTTP request would time out. So we return "processing" instantly and let the client poll — this is the standard pattern for slow jobs.

---

5. Query pipeline (walk-through)

Code path: `routes_query.py` → `query_engine.py` → `indexing.py`

1. `POST /api/query` with `{session_id, question}`.
2. **Retrieve** (`query_engine.answer_question`): the question is embedded, and Qdrant returns the `top_k` most similar chunks — **filtered to this claim** via a metadata filter on `claim_id`. (This is why one claim's documents never leak into another's answers — verified in testing.)
3. **Synthesize**: LlamaIndex feeds those chunks + the question to the Gemma model with a system prompt that says *"answer using ONLY this context."* This is what keeps answers grounded and stops hallucination.
4. **Citations**: we read them **structurally** from the retrieved nodes' metadata (`filename`, `page 2`, similarity `score`) — *not* by asking the LLM to cite itself. This is more reliable and is the honest proof that the answer came from the documents.
5. **Confidence**: reported as the top chunk's similarity score. It's a *retrieval-confidence proxy*, not a claims-decision certainty — deliberately, because this is advisory Q&A, not an adjudication engine.

---

6. The provider abstraction (why it matters)

`services/llm_provider.py` is the **only** file that knows we use OpenRouter. Everything else asks it for "an LLM" or "an embedding model" via a small interface (`LLMProvider`). Benefits:

- Swap providers (OpenAI, a local model) by editing one file.
- The phase-2 LangChain agent layer plugs into the same place.

`services/openrouter_embedding.py` exists for a specific reason: the standard OpenAI embedding client defaults to `encoding_format="base64"`, which OpenRouter returns as null → a crash. Our class forces `encoding_format="float"`. (A good example of a real-world integration gotcha you only find by testing.)

---

7. API reference

Method	Path	Purpose
POST	`/api/sessions`	Start a claim; returns `session_id`
POST	`/api/documents?session_id=`	Upload file(s); starts background ingestion
GET	`/api/documents/{doc_id}/status`	Poll ingestion status
GET	`/api/sessions/{session_id}/documents`	List a claim's documents
POST	`/api/query`	Ask a question; returns answer + citations
GET	`/api/health`	Liveness + Qdrant reachability

Interactive docs auto-generated at `http://localhost:8000/docs` when running.

---

8. What's next (phase 2)

These were deliberately deferred to ship a working core first:

- **Agentic routing (LangChain):** a router that picks a strategy per question (semantic search vs. whole-doc summary vs. policy comparison).
- **Hybrid search:** combine keyword (BM25) + vector search for better recall.
- **Eval harness:** ~20–30 Q&A pairs measuring retrieval precision/recall across chunking strategies — the artifact that *proves* accuracy to a reviewer.

---

9. Glossary

- **Embedding** — a vector representation of text; similar meanings → nearby vectors.
- **Chunk / Node** — a bounded piece of a document that gets embedded and stored.
- **Vector store** — a database optimized for "find the nearest vectors" (Qdrant).
- **Retrieval** — finding the most relevant chunks for a query.
- **Grounding** — forcing the answer to come from retrieved context, not the model's memory.
- **Citation** — the source (file + page + score) a chunk came from.